

00_SETUP_Y_EJEMPLO_INICIAL

August 21, 2026



1 MÓDULO 0: Configuración del Entorno de Trabajo y Ejemplo Práctico Integrador (Titanic Dataset)

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora 30 minutos

Módulo: 00 - Environment Setup & Integrated Titanic ML Pipeline

1.1 1. Objetivos de Aprendizaje

1. **Entorno de Trabajo Reproducible:** Verificar la instalación de Python 3.12+, la configuración de **Antigravity-IDE**, la creación del entorno virtual (**venv**) y el registro del kernel de Jupyter (**ipykernel**).
 2. **Buenas Prácticas en Ciencia Abierta:** Organizar un repositorio estructurado, aislar dependencias con **requirements.txt** y asegurar la ejecución en entornos locales y remotos.
 3. **Primer Contacto Hands-on:** Cargar, inspeccionar, limpiar, enriquecer y visualizar un conjunto de datos real (*Titanic Dataset*) combinando **pandas**, **matplotlib**, **seaborn** y un modelo inicial de **scikit-learn**.
-

1.2 2. Preparación y Verificación del Entorno de Trabajo

Para garantizar que los cuadernos del curso se ejecuten sin errores:

1.2.1 Pasos de Configuración:

1. Crear y activar el entorno virtual:

```
# En macOS/Linux
python3 -m venv venv
source venv/bin/activate
```

```
# En Windows (PowerShell)
python -m venv venv
.\venv\Scripts\Activate.ps1
```

2. Instalar dependencias del curso:

```
pip install --upgrade pip
pip install -r requirements.txt
```

3. Registrar el entorno virtual como kernel de Jupyter:

```
python -m ipykernel install --user --name=curso_ciencia_abierta --display-name "CURSO Python Ciencia Abierta"
```

4. Seleccionar el Kernel en Antigravity-IDE:

- En la esquina superior derecha del cuaderno, haz clic en **Kernel** y selecciona “**CURSO Python Ciencia Abierta**” (o el intérprete `./venv/bin/python`).

1.3 3. Importaciones y Configuración del Estilo Visual

Importamos las librerías fundamentales de Ciencia de Datos y Machine Learning, configurando un estilo gráfico profesional.

```
[ ]: import os
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Configuración del estilo gráfico para visualizaciones científicas
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'Helvetica'
plt.rcParams['axes.edgecolor'] = '#cccccc'
plt.rcParams['axes.linewidth'] = 0.8
```

```

print("=== VERIFICACIÓN DEL ENTORNO ===")
print(f"Ejecutable de Python: {sys.executable}")
print(f"Versión de Python: {sys.version.split()[0]}")
print(f"Versión de Pandas: {pd.__version__}")

```

1.4 4. Carga del Dataset Titanic desde la Carpeta Raíz DATASETS/

Cargamos los datos del Titanic desde la carpeta oficial DATASETS/titanic/train.csv con resolución dinámica multinivel.

```

[ ]: import os
cur_dir = os.getcwd()
# -----
# 1. Carga del conjunto de TRAIN
# -----
train_path = None
for attempt in [
    os.path.join(cur_dir, "DATASETS", "titanic", "train.csv"),
    os.path.join(cur_dir, "..", "DATASETS", "titanic", "train.csv"),
    os.path.join(cur_dir, "..", "..", "DATASETS", "titanic", "train.csv"),
    os.path.join(cur_dir, "..", "..", "..", "DATASETS", "titanic", "train.csv")
]:
    if os.path.exists(attempt):
        train_path = os.path.abspath(attempt)
        break
if train_path and os.path.exists(train_path):
    train_df = pd.read_csv(train_path)
    print(f"Dataset de train cargado correctamente desde: {train_path}")
# -----
# 2. Carga del conjunto de TEST
# -----
test_path = None
for attempt in [
    os.path.join(cur_dir, "DATASETS", "titanic", "test.csv"),
    os.path.join(cur_dir, "..", "DATASETS", "titanic", "test.csv"),
    os.path.join(cur_dir, "..", "..", "DATASETS", "titanic", "test.csv"),
    os.path.join(cur_dir, "..", "..", "..", "DATASETS", "titanic", "test.csv")
]:
    if os.path.exists(attempt):
        test_path = os.path.abspath(attempt)
        break
if test_path and os.path.exists(test_path):
    test_df = pd.read_csv(test_path) # Corregido: pasar test_path en vez de_
    ↪ test_df
    print(f"Dataset de test cargado correctamente desde: {test_path}")

```

```
# -----
# 3. Concatenación vertical (axis=0) de TRAIN y TEST
# -----
# Corregido: pasar lista [train_df, test_df], axis=0 para apilar filas e
  ↳ ignore_index=True (con True en mayúscula)
df = pd.concat([train_df, test_df], axis=0, ignore_index=True)
print(f"\nDimensiones del dataset unificado: {df.shape[0]} filas x {df.
  ↳ shape[1]} columnas")
print("Primeros 5 registros:")
print(df.head(5))
```

1.5 5. Limpieza e Imputación de Datos Faltantes

Imputamos la mediana para la edad (Age) y la moda para el puerto de embarque (Embarked).

```
[ ]: print("Valores nulos antes de la limpieza:")
print(df[["Age", "Embarked", "Cabin"]].isnull().sum())

df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

print("Valores nulos tras la imputación:")
print(df[["Age", "Embarked"]].isnull().sum())
print("Imputación completada con éxito.")

#Salvamos el dataset limpio
```

1.6 6. Visualización Científica 1: Tasa de Supervivencia por Clase de Pasaje (Pclass)

Representamos la tasa porcentual de supervivencia según la clase de pasaje con barras estilizadas.

```
[ ]: fig, ax = plt.subplots(figsize=(9, 5))
pclass_survival = df.groupby(["Pclass", "Survived"]).size().
  ↳ unstack(fill_value=0)
pclass_pct = pclass_survival.div(pclass_survival.sum(axis=1), axis=0) * 100

bars = pclass_pct.plot(kind="bar", stacked=False, ax=ax, color=["#d9534f",
  ↳ "#2ca02c"], width=0.7)
ax.set_title("Proporción de Supervivencia por Clase de Pasaje (Titanic)",
  ↳ fontsize=13, fontweight='bold', pad=15)
ax.set_xlabel("Clase de Pasaje", fontsize=11, labelpad=10)
```

```

ax.set_ylabel("Porcentaje de Pasajeros (%)", fontsize=11)
ax.set_xticklabels(["1ª Clase", "2ª Clase", "3ª Clase"], rotation=0)
ax.legend(["Fallecido (0)", "Superviviente (1)"], title="Estado", loc="upper_
↪right")
ax.set_ylim(0, 100)

for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.annotate(f"{height:.1f}%", (p.get_x() + p.get_width() / 2., height /
↪2.),
                    ha='center', va='center', fontsize=10, color='white',
↪fontweight='bold')

plt.tight_layout()
#plt.close(fig)
print(" Gráfico de barras de supervivencia por clase generado.")

```

1.7 7. Visualización Científica 2: Dispersión Edad vs Tarifa Pagada

Analizamos la relación entre la edad del pasajero, la tarifa pagada y el resultado de supervivencia.

```

[ ]: fig, ax = plt.subplots(figsize=(10, 6))
scatter = ax.scatter(df["Age"], np.log2(df["Fare"]), c=df["Survived"],
↪cmap="RdYlGn", alpha=0.7, edgecolors='w', s=50)

ax.set_title("Relación entre Edad, Tarifa Pagada (Escala Logarítmica) y
↪Supervivencia", fontsize=13, fontweight='bold', pad=15)
ax.set_xlabel("Edad del Pasajero (Años)", fontsize=11)
ax.set_ylabel("Tarifa del Pasaje (UK Libra - Escala Log2)", fontsize=11)
cbar = plt.colorbar(scatter, ax=ax, ticks=[0, 1])
cbar.ax.set_yticklabels(['Fallecido (0)', 'Sobreviviente (1)'])

plt.tight_layout()
#plt.close(fig)
print(" Diagrama de dispersión Edad vs Tarifa generado.")

```

1.8 8. Visualización Científica 3: Gráfico Donut de Supervivencia por Sexo

Visualizamos mediante un gráfico de tipo Donut la marcada diferencia en la tasa de supervivencia según el género del pasajero.

```

[ ]: survival_by_sex = df.groupby("Sex")["Survived"].mean() * 100

```

```

fig, ax = plt.subplots(figsize=(7, 7))
colors_sex = ["#ff9999", "#66b3ff"]
wedges, texts, autotexts = ax.pie(
    survival_by_sex,
    labels=["Mujeres (female)", "Hombres (male)"],
    autopct='%1.1f%%',
    startangle=90,
    colors=colors_sex,
    wedgeprops=dict(width=0.4, edgecolor='w')
)

plt.setp(autotexts, size=11, weight="bold", color="black")
ax.set_title("Tasa de Supervivencia según Sexo (%)", fontsize=13,
    ↪fontweight='bold', pad=30)

plt.tight_layout()
#plt.close(fig)
print(" Gráfico donut de supervivencia por sexo generado.")

```

1.9 9. Ingeniería de Variables y Análisis de Grupos Familiares

Creamos nuevas variables sintéticas: FamilySize (tamaño del grupo familiar) e IsAlone (si el pasajero viajaba en solitario).

```

[ ]: df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df = df[df["FamilySize"] < 8]
family_stats = df.groupby("FamilySize")["Survived"].agg(["count", "mean"]).
    ↪rename(columns={"count": "Pasajeros", "mean": "Tasa Supervivencia"})
print("Supervivencia media según el tamaño de la familia:")
print(family_stats)

# Visualización gráfica de la Tasa de Supervivencia según el Tamaño Familiar
fig, ax = plt.subplots(figsize=(9, 5))
family_pct = family_stats["Tasa Supervivencia"] * 100

bars = family_pct.plot(kind="bar", ax=ax, color="#2b5c8f", width=0.65)
ax.set_title("Tasa de Supervivencia según el Tamaño del Grupo Familiar",
    ↪fontsize=13, fontweight="bold", pad=15)
ax.set_xlabel("Tamaño de la Familia (Pasajero + Parientes)", fontsize=11,
    ↪labelpad=10)
ax.set_ylabel("Tasa de Supervivencia (%)", fontsize=11)
ax.set_xticklabels(family_stats.index, rotation=0)
ax.set_ylim(0, 100)

for p, (_, row) in zip(ax.patches, family_stats.iterrows()):

```

```

height = p.get_height()
count = int(row["Pasajeros"])
if height > 0:
    ax.annotate(f"{height:.1f}%\n(n={count})", (p.get_x() + p.get_width() / 2., height + 2),
                ha="center", va="bottom", fontsize=8.5, fontweight="bold",
                color="#1f2d3d")

plt.tight_layout()
# plt.close(fig)
print(" Gráfico de tasa de supervivencia por tamaño de familia generado.")

```

1.10 10. Modelo Inicial de Machine Learning e Importancia de Variables

Entrenamos un modelo RandomForestClassifier e inspeccionamos qué atributos resultaron más determinantes para predecir la supervivencia.

```

[ ]: # 1. Filtrar solo las filas con etiqueta de supervivencia (omitir filas de test
    con Survived NaN)
df_model = df.dropna(subset=["Survived"]).copy()
# 2. Imputación de seguridad para Fare por si hubiera algún nulo
df_model["Fare"] = df_model["Fare"].fillna(df_model["Fare"].median())
# 3. Selección de características X y objetivo y
feature_cols = ["Pclass", "Age", "SibSp", "Parch", "Fare", "FamilySize",
    "IsAlone"]
X = pd.get_dummies(df_model[feature_cols + ["Sex", "Embarked"]],
    drop_first=True)
y = df_model["Survived"].astype(int)
# 4. División Train / Test y Entrenamiento del Modelo
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
# 5. Importancia de Variables y Graficado
importances = pd.Series(model.feature_importances_, index=X.columns).
    sort_values(ascending=True)
fig, ax = plt.subplots(figsize=(9, 6))
importances.plot(kind="barh", ax=ax, color="#2b5c8f")
ax.set_title("Importancia de Variables en la Predicción de Supervivencia
    (Random Forest)", fontsize=13, fontweight='bold', pad=15)
ax.set_xlabel("Importancia Relativa del Atributo", fontsize=11)
ax.set_ylabel("Variables del Modelo", fontsize=11)
plt.tight_layout()
plt.show()

```

```
print(f"Precisión del modelo RandomForest en test: {model.score(X_test, y_test)*100:.2f}%")
print(" Gráfico de Importancia de Variables generado.")
```

2 Visualización adicional:

Eje X: Código único del pasajero (PassengerId del 892 al 1309). Eje Y1 (Azul / Izquierda): Supervivencia Real (Ground Truth de gender_submission.csv). Eje Y2 (Naranja / Derecha): Predicción emitida por el árbol RandomForestClassifier. Métrica Impresa: Precisión del 84.45% alcanzada sobre los 418 pasajeros de prueba.

```
[ ]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from matplotlib.lines import Line2D

# -----
# 1. Cargar gender_submission.csv y preparar el conjunto de TEST
# -----

test_data = df[df["Survived"].isnull()].copy()

gender_path = None
for attempt in [
    os.path.join(cur_dir, "DATASETS", "titanic", "gender_submission.csv"),
    os.path.join(cur_dir, "..", "DATASETS", "titanic", "gender_submission.csv"),
    os.path.join(cur_dir, "..", "..", "DATASETS", "titanic", "gender_submission.
↪csv"),
    os.path.join(cur_dir, "..", "..", "..", "DATASETS", "titanic",
↪"gender_submission.csv")
]:
    if os.path.exists(attempt):
        gender_path = os.path.abspath(attempt)
        break

if gender_path and os.path.exists(gender_path):
    gender_df = pd.read_csv(gender_path)
    test_data = test_data.drop(columns=["Survived"]).merge(gender_df,
↪on="PassengerId")

# Imputación de nulos remanentes en test (Age y Fare)
test_data["Age"] = test_data["Age"].fillna(df_model["Age"].median())
test_data["Fare"] = test_data["Fare"].fillna(df_model["Fare"].median())
test_data["Embarked"] = test_data["Embarked"].fillna("S")
```

```

# Recrear la matriz de características alineada exactamente con X
X_test_set = pd.get_dummies(test_data[feature_cols + ["Sex", "Embarked"]],
    ↪drop_first=True)
X_test_set = X_test_set.reindex(columns=X.columns, fill_value=0)

# -----
# 2. Inferencia: Predicciones y Probabilidades del Modelo
# -----
test_data["Prediction"] = model.predict(X_test_set)
test_data["Prob_Survival"] = model.predict_proba(X_test_set)[: , 1] #
    ↪Probabilidad estimada de sobrevivir
test_data["EsAcierto"] = test_data["Survived"] == test_data["Prediction"]

acc_test = test_data["EsAcierto"].mean() * 100

# -----
# 3. Visualización Científica Integrada (Matriz de Confusión + Probabilidades)
# -----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5.5),
    ↪gridspec_kw={"width_ratios": [1, 2]})

# Panel 1: Matriz de Confusión Térmica
cm = confusion_matrix(test_data["Survived"], test_data["Prediction"])
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax1,
    xticklabels=["Fallecido (0)", "Sobreviviente (1)",
    yticklabels=["Fallecido (0)", "Sobreviviente (1)",
    annot_kws={"size": 12, "weight": "bold"})
ax1.set_title("Matriz de Confusión (Test Set)", fontsize=12, fontweight="bold",
    ↪pad=12)
ax1.set_xlabel("Predicción del Modelo", fontsize=10)
ax1.set_ylabel("Valor Real (Ground Truth)", fontsize=10)

# Panel 2: Probabilidad Predicha por Pasajero (PassengerId) y Estado de
    ↪Clasificación
colors = test_data["EsAcierto"].map({True: "#2ca02c", False: "#d62728"})
ax2.scatter(test_data["PassengerId"], test_data["Prob_Survival"], c=colors,
    ↪alpha=0.75, s=30)
ax2.axhline(0.5, color="#333333", linestyle="--", linewidth=1.2, label="Umbral
    ↪de Decisión (p = 0.5)")

ax2.set_title(f"Probabilidad de Supervivencia Estimada por Pasajero (Precisión:
    ↪{acc_test:.2f}%)",
    fontsize=12, fontweight="bold", pad=12)
ax2.set_xlabel("Código de Pasajero (PassengerId)", fontsize=10)
ax2.set_ylabel("Probabilidad Estimada de Supervivencia", fontsize=10)

```

```

ax2.set_ylim(-0.05, 1.05)

# Leyenda personalizada para Aciertos vs Errores
legend_elements = [
    Line2D([0], [0], marker="o", color="w", label="Acierto (Clasificación ↵
↵Correcta)", markerfacecolor="#2ca02c", markersize=8),
    Line2D([0], [0], marker="o", color="w", label="Error (Falso Positivo / ↵
↵Negativo)", markerfacecolor="#d62728", markersize=8),
    Line2D([0], [0], color="#333333", lw=1.2, linestyle="--", label="Umbral p = ↵
↵0.5")
]
ax2.legend(handles=legend_elements, loc="upper right", fontsize=9)

plt.tight_layout()
plt.show()

print(f" Evaluación completada sobre {len(test_data)} pasajeros de test.")
print(f" Precisión Global en el Test Set: {acc_test:.2f}%")

```

- 3 Por último vamos a buscar pasajeros afortunados y desafortunados. Aquellos con probabilidad de sobrevivir inferior al 20% y que sobrevivieron y aquellos con probabilidad de sobrevivir superior al 80% que murieron.

```

[ ]: # -----
# 1. Preparar Dataset Completo (1.309 Pasajeros) con Ground Truth Unificado
# -----
# Unificar train + test + gender_submission en un único DataFrame completo
gender_path = None
for attempt in [
    os.path.join(cur_dir, "DATASETS", "titanic", "gender_submission.csv"),
    os.path.join(cur_dir, "..", "DATASETS", "titanic", "gender_submission.csv"),
    os.path.join(cur_dir, "..", "..", "DATASETS", "titanic", "gender_submission.
↵csv"),
    os.path.join(cur_dir, "..", "..", "..", "DATASETS", "titanic", ↵
↵"gender_submission.csv")
]:
    if os.path.exists(attempt):
        gender_path = os.path.abspath(attempt)
        break

df_full = df.copy()
if gender_path and os.path.exists(gender_path):
    gender_df = pd.read_csv(gender_path)
    # Completar los valores Survived faltantes con gender_submission

```

```

df_full = df_full.set_index("PassengerId")
gender_df = gender_df.set_index("PassengerId")
df_full["Survived"] = df_full["Survived"].fillna(gender_df["Survived"]).
↳reset_index()["Survived"].values
df_full = df_full.reset_index()

# Imputación de seguridad sobre todo el dataset
df_full["Age"] = df_full["Age"].fillna(df_full["Age"].median())
df_full["Fare"] = df_full["Fare"].fillna(df_full["Fare"].median())
df_full["Embarked"] = df_full["Embarked"].fillna("S")

# Recrear características X_all para los 1.309 registros
feature_cols = ["Pclass", "Age", "SibSp", "Parch", "Fare", "FamilySize",
↳"IsAlone"]
X_all = pd.get_dummies(df_full[feature_cols + ["Sex", "Embarked"]],
↳drop_first=True)
X_all = X_all.reindex(columns=X.columns, fill_value=0)

# -----
# 2. Inferencia sobre los 1.309 Pasajeros
# -----
df_full["Prediction"] = model.predict(X_all)
df_full["Prob_Survival"] = model.predict_proba(X_all)[:, 1]

# Identificar Afortunados (< 20% prob y Sobrevivió) y Desafortunados (> 80%
↳prob y Falleció)
lucky_all = df_full[(df_full["Prob_Survival"] < 0.20) & (df_full["Survived"] ==
↳1)].sort_values("Prob_Survival")
unlucky_all = df_full[(df_full["Prob_Survival"] > 0.80) & (df_full["Survived"]
↳== 0)].sort_values("Prob_Survival", ascending=False)

# -----
# 3. Imprimir Tablas de Resultados
# -----
cols_resumen = ["PassengerId", "Name", "Pclass", "Sex", "Age", "Fare",
↳"FamilySize", "Prob_Survival"]

print(f"=== ANÁLISIS GLOBAL SOBRE LOS {len(df_full)} PASAJEROS DE A BORDO ===")
N_lucky = len(lucky_all)
print(f" - Pasajeros Afortunados (< 20% probabilidad y SOBREVIVIÓ): {N_lucky}")

N_unlucky = len(unlucky_all)
print(f" - Pasajeros Desafortunados (> 80% probabilidad y FALLECIÓ):
↳{N_unlucky}")

print("\n--- TOP PASAJEROS AFORTUNADOS (Supervivencia Inesperada) ---")

```

```

print(lucky_all[cols_resumen].head(N_lucky).to_string(index=False))

print("\n--- TOP PASAJEROS DESAFORTUNADOS (Fallecimiento Inesperado) ---")
print(unlucky_all[cols_resumen].head(N_unlucky).to_string(index=False))

# -----
# 4. Visualización Gráfica Unificada sobre los 1.309 Pasajeros
# -----
fig, ax = plt.subplots(figsize=(13, 6))

# Plotear pasaje general
ax.scatter(df_full["PassengerId"], df_full["Prob_Survival"],
           color="#cccccc", alpha=0.35, s=20, label=f"Pasaje General",
           ↪(n={len(df_full)}))

# Destacar Afortunados en Verde y Desafortunados en Rojo
ax.scatter(lucky_all["PassengerId"], lucky_all["Prob_Survival"],
           color="#2ca02c", alpha=0.9, s=75, edgecolors="k", zorder=5,
           label=f"Pasajeros Afortunados (<20% prob | n={len(lucky_all)}))")

ax.scatter(unlucky_all["PassengerId"], unlucky_all["Prob_Survival"],
           color="#d62728", alpha=0.9, s=75, edgecolors="k", zorder=5,
           label=f"Pasajeros Desafortunados (>80% prob | n={len(unlucky_all)}))")

# Umbrales
ax.axhline(0.20, color="#2ca02c", linestyle="--", linewidth=1.2, label="Umbral",
           ↪20% (Afortunados)")
ax.axhline(0.80, color="#d62728", linestyle="--", linewidth=1.2, label="Umbral",
           ↪80% (Desafortunados)")

# Anotar apellidos de los 4 casos más extremos de cada grupo
for _, row in lucky_all.head(4).iterrows():
    surname = row["Name"].split(",")[0]
    ax.annotate(surname, (row["PassengerId"], row["Prob_Survival"]),
               xytext=(0, 10), textcoords="offset points", ha="center",
               ↪fontsize=8.5, fontweight="bold", color="#1e7e34")

for _, row in unlucky_all.head(4).iterrows():
    surname = row["Name"].split(",")[0]
    ax.annotate(surname, (row["PassengerId"], row["Prob_Survival"]),
               xytext=(0, -14), textcoords="offset points", ha="center",
               ↪fontsize=8.5, fontweight="bold", color="#a94442")

ax.set_title(f"Análisis Global del Pasaje (1.309 Pasajeros): Pasajeros",
           ↪Afortunados vs Desafortunados",
           fontsize=13, fontweight="bold", pad=15)

```

```

ax.set_xlabel("Código de Pasajero (PassengerId)", fontsize=11)
ax.set_ylabel("Probabilidad de Supervivencia Estimada por el Modelo",
↪ fontsize=11)
ax.set_ylim(-0.03, 1.03)
ax.legend(loc="center right", fontsize=9.5)

plt.tight_layout()
plt.show()

```

3.1 11. Ejercicios Prácticos de Autoevaluación

3.1.1 Ejercicio 1:

Calcula la tasa de supervivencia de los pasajeros que viajaban solos (`IsAlone == 1`) frente a los que viajaban acompañados (`IsAlone == 0`).

```

[ ]: # SOLUCIÓN EJERCICIO 1:
alone_survival = df.groupby("IsAlone")["Survived"].mean() * 100
print("Tasa de Supervivencia Solo (1) vs Acompañado (0):")
print(f" - Acompañado: {alone_survival[0]:.2f}%")
print(f" - Solo:      {alone_survival[1]:.2f}%")

```

3.1.2 Ejercicio 2:

Filtra los pasajeros con una tarifa (`Fare`) superior a \$100 y calcula su tasa de supervivencia.

```

[ ]: # SOLUCIÓN EJERCICIO 2:
high_fare = df[df["Fare"] > 100]
print(f"Pasajeros con tarifa superior a $100: {len(high_fare)}")
print(f"Tasa de supervivencia en tarifa alta: {high_fare['Survived'].mean() *
↪ 100:.2f}%")

```